Vue.js kódmagyarázó segédlet

Kódblokkok magyarázata, összekötések, módosítási minták és gyakori hibák frontend feladatokhoz.

1. Alap gondolkodás Vue.js-ben

A Vue frontend lényege: komponensekre bontod az oldalt. Egy komponens általában három részből áll: template, script és style. A template a HTML-szerű megjelenés, a script az adatok és függvények helye, a style pedig a dizájn.

Feladatnál először mindig ezt döntsd el: milyen adatok vannak, mit kell listázni, mit kell űrlapon bekérni, kell-e API hívás, és melyik rész legyen külön komponens.

> Mit kell változtatni? Ha más adat kell, a cim/leiras helyére saját változókat írsz. Ha más gombművelet kell, a mentes() függvény tartalmát cseréled.

```
// Alap komponens felépítés
<template>
  <div class="card">
    <h2>{{ cim }}</h2>
    <p>{{ leiras }}</p>

    <button @click="mentes">
      Mentés
    </button>
  </div>
</template>

<script setup>
import { ref } from 'vue'

const cim = ref('Kezdő cím')
const leiras = ref('Ide jön a szöveg')

function mentes() {
  console.log('Mentés gomb megnyomva')
}
</script>

<style scoped>
.card {
  padding: 20px;
  border-radius: 12px;
  background: white;
}
</style>
```

2. Telepítés és indítás

Vue projektet legegyszerűbben Vite-tal érdemes indítani. A parancsokat VS Code terminálba vagy sima terminálba is írhatod. A projekt mappájában dolgozz.

> Várt eredmény: a terminál kiír egy localhost címet, például http://localhost:5173/. Ha npm nem ismert, Node.js nincs telepítve vagy nincs újraindítva a terminál.

```
# VS Code terminál
npm create vue@latest frontend

# Belépés a projektbe
cd frontend

# Csomagok telepítése
npm install

# Fejlesztői szerver indítása
npm run dev
```

3. Composer és Vue kapcsolata

A Composer nem Vue csomagkezelő, hanem PHP/Laravel oldalhoz tartozik. Vue-hoz npm kell. Laravel + Vue projektben viszont mindkettő szerepelhet: composer install a backendhez, npm install a frontendhez.

> Ha Laravel projektben Vue is van, általában két szervert futtatsz fejlesztéskor: php artisan serve és npm run dev.

```
# Laravel oldal
composer install
php artisan key:generate
php artisan migrate

# Frontend/Vite oldal
npm install
npm run dev
```

4. Mappastruktúra - mit hova írj?

Hely	Mire való?	Mikor módosítod?
src/main.js	Vue app indítása	Ha plugint, routert, Bootstrapot, Pinia-t kötsz be
src/App.vue	Fő komponens	Ha az egész oldal keretét építed
src/components	Újrafelhasználható komponensek	Kártya, lista, navbar, űrlap
src/views	Oldalak	Home, Login, Dashboard, Részletek oldal
src/router/index.js	Útvonalak	Ha több oldal között navigálsz
src/assets	Képek, CSS	Globális stílusok, logó, háttér

5. Lista megjelenítése v-for segítségével

Ha a feladat több rekordot kér megjeleníteni, szinte biztosan v-for kell. A lista lehet kézzel létrehozott tömb vagy API-ból érkező adat.

> Ha nem termékek vannak, csak átnevezed: termekek -> autok, tanulok, ingatlanok, turak stb. A mezőneveknek egyezniük kell az adataiddal.

```
<template>
  <div>
    <h1>Termékek</h1>

    <div v-for="termek in termekek" :key="termek.id" class="termek-card">
      <h3>{{ termek.nev }}</h3>
      <p>Ár: {{ termek.ar }} Ft</p>
      <p>Kategória: {{ termek.kategoria }}</p>
    </div>
  </div>
</template>

<script setup>
import { ref } from 'vue'

const termekek = ref([
  { id: 1, nev: 'Laptop', ar: 250000, kategoria: 'Elektronika' },
  { id: 2, nev: 'Egér', ar: 8000, kategoria: 'Kiegészítő' }
])
</script>
```

6. Űrlap készítése v-modellel

Bekéréshez v-model kell. Ez összeköti az input mezőt a JavaScript változóval. Amit a felhasználó beír, az azonnal bekerül a változóba.

Szöveghez sima v-model, számhoz v-model.number, checkboxhoz boolean változó kell.

```
<template>
  <form @submit.prevent="kuldes">
    <label>Név</label>
    <input v-model="form.nev" type="text">

    <label>Ár</label>
    <input v-model.number="form.ar" type="number">

    <label>Aktív</label>
    <input v-model="form.aktiv" type="checkbox">

    <button type="submit">Küldés</button>
  </form>
</template>

<script setup>
import { reactive } from 'vue'

const form = reactive({
  nev: '',
  ar: 0,
  aktiv: false
})

function kuldes() {
  console.log(form)
}
</script>
```

7. API hívás Laravel backend felé

Ha a frontend Laravel API-ból kapja az adatot, fetch vagy axios kell. Kezdésnek a fetch is elég. A lényeg: mounted állapotban lekéred az adatokat, majd betöltöd egy ref változóba.

Ha más endpoint kell, csak az URL-t cseréled. Ha a backend más mezőnevet ad vissza, a template-ben is azt kell használni.

```
<template>
  <div>
    <h1>Adatok</h1>

    <p v-if="loading">Betöltés...</p>

    <div v-for="item in adatok" :key="item.id">
      {{ item.nev }}
    </div>
  </div>
</template>

<script setup>
import { ref, onMounted } from 'vue'

const adatok = ref([])
const loading = ref(true)

onMounted(async () => {
  try {
    const response = await fetch('http://127.0.0.1:8000/api/kategoriak')
    adatok.value = await response.json()
  } catch (error) {
    console.error('Hiba az adatok lekérésekor:', error)
  } finally {
    loading.value = false
  }
})
</script>
```

8. POST kérés - adat mentése

Ha űrlapból menteni kell adatot, POST kérést küldesz. A headers részben megadod, hogy JSON-t küldesz, a body-ban pedig átalakítod az objektumot JSON szöveggé.

Ha 422 hibát kapsz, Laravel validáció dobta vissza. Ilyenkor a beküldött mezőnevek vagy kötelező mezők nem stimmelnek.

```
async function ment() {
  const response = await fetch('http://127.0.0.1:8000/api/termekek', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Accept': 'application/json'
    },
    body: JSON.stringify(form)
  })

  if (!response.ok) {
    const hibak = await response.json()
    console.log('Validációs hiba:', hibak)
    return
  }

  const ujElem = await response.json()
  termekek.value.push(ujElem)
}
```

9. Feltételes megjelenítés

v-if akkor kell, ha valamit csak bizonyos feltételnél akarsz mutatni. Például aktív/inaktív státusz, betöltés, hibaüzenet, üres lista.

Optimalizálás: először mindig kezeld a loading állapotot, utána a hibát, utána az üres listát, végül a normál adatot.

```
<p v-if="loading">Betöltés...</p>

<p v-else-if="hiba">
  Hiba történt: {{ hiba }}
</p>

<p v-else-if="adatok.length === 0">
  Nincs megjeleníthető adat.
</p>

<div v-else>
  <div v-for="adat in adatok" :key="adat.id">
    {{ adat.nev }}
  </div>
</div>
```

10. Keresés és szűrés

Ha a feladat keresést kér, computed változóval érdemes szűrni. Így nem módosítod az eredeti listát, csak a megjelenített listát.

Ha név helyett cím, kategória vagy leírás alapján kell keresni, az item.nev részt cseréled.

```
<template>
  <input v-model="kereses" placeholder="Keresés...">

  <div v-for="item in szurtLista" :key="item.id">
    {{ item.nev }}
  </div>
</template>

<script setup>
import { ref, computed } from 'vue'

const kereses = ref('')
const lista = ref([
  { id: 1, nev: 'Budapest' },
  { id: 2, nev: 'Szeged' },
  { id: 3, nev: 'Kecskemét' }
])

const szurtLista = computed(() => {
  return lista.value.filter(item =>
    item.nev.toLowerCase().includes(kereses.value.toLowerCase())
```

```
  )
})
</script>
```

11. Komponens bontás - mikor kell külön komponens?

Ha ugyanaz a megjelenítés többször kell, legyen külön komponens. Például ProductCard.vue, UserRow.vue, Navbar.vue, FormInput.vue.

> Props = adat átadása szülőtől gyereknek. Emit = esemény visszaküldése gyerektől szülőnek.

```
<!-- components/TermekKartya.vue -->
<template>
  <div class="card">
    <h3>{{ termek.nev }}</h3>
    <p>{{ termek.ar }} Ft</p>
    <button @click="$emit('torles', termek.id)">Törlés</button>
  </div>
</template>

<script setup>
defineProps({
  termek: {
    type: Object,
    required: true
  }
})

defineEmits(['torles'])
</script>
```

12. Szülő komponens a kártyákhoz

> Ha más típusú elemek vannak, csak a komponens nevét és a mezőket cseréled. A logika ugyanaz.

```
<template>
  <TermekKartya
    v-for="termek in termekek"
    :key="termek.id"
    :termek="termek"
    @torles="torol"
  />
</template>

<script setup>
import { ref } from 'vue'
import TermekKartya from './components/TermekKartya.vue'

const termekek = ref([
  { id: 1, nev: 'Monitor', ar: 60000 }
])

function torol(id) {
  termekek.value = termekek.value.filter(t => t.id !== id)
}
</script>
```

13. Dizájn alapelvek Vue projektnél

Kérés	Megoldás
Lista kártyákkal	display: grid; gap; border-radius; box-shadow
Űrlap rendezett legyen	label + input blokk, max-width, margin auto
Mobilon is jó legyen	media query vagy Bootstrap grid
Aktív/inaktív státusz	class binding: :class
Betöltés közben	loading szöveg vagy spinner
Hiba esetén	piros alert blokk

14. Dinamikus class használata

Ha más feltétel alapján kell színezni, az item.aktiv helyére más logikai feltételt írsz.

```
<div
  class="statusz"
  :class="item.aktiv ? 'aktiv' : 'inaktiv'"
>
  {{ item.aktiv ? 'Aktív' : 'Nem aktív' }}
</div>

<style scoped>
.statusz {
  padding: 8px 12px;
  border-radius: 8px;
  font-weight: bold;
}

.aktiv {
  background: #d1fae5;
  color: #065f46;
}

.inaktiv {
  background: #fee2e2;
  color: #991b1b;
}
</style>
```

15. Gyakori hibák és javítások

Hiba	Ok	Javítás
npm nem ismert	Node.js nincs telepítve	Telepíts Node.js-t, majd nyisd újra a terminált
Üres lista jelenik meg	API nem ad adatot vagy rossz mezőnév	Nézd meg console.log(adatok.value)
CORS hiba	Frontend és backend más portról fut	Laravel CORS beállítás vagy azonos domain
Cannot read property	Még nincs betöltve az adat	Használj v-if-et
Component not found	Import vagy fájlnév hibás	Ellenőrizd az import útvonalat